AGENTS.md

Guidance for automated coding agents that add features or maintain `mdexplore`.

Mission

Maintain a fast, reliable Markdown explorer for Ubuntu/Linux desktop with:

- Left-pane directory tree rooted at a target folder.
- Markdown-only file listing (`*.md`).
- Right-pane rendered preview with math and diagram support.
- `^`, `Refresh`, `PDF`, `Add View`, and `Edit` actions.
- Top-right copy-by-color controls for clipboard file operations.
- A pin button before copy-by-color controls to copy the currently previewed file.

Repository Map

- `mdexplore.py` : main application (Qt UI, renderer, file loading, cache, CLI path arg).
- `mdexplore.sh` : launcher (venv lifecycle + dependency install + app run).
- `requirements.txt` : Python runtime dependencies.
- `README.md` : user-facing setup and usage documentation.
- `RENDER-PATHS.md` : detailed render/caching architecture map (Mermaid diagrams + prose).
- `mdexplore.desktop.sample` : user-adaptable `.desktop` launcher template.
- `mdexplor-icon.png` : primary app icon asset (preferred).
- `DESCRIPTION.md` : short repository summary and suggested topic tags.
- `LICENSE` : MIT license text.
- Runtime config file: `~/.mdexplore.cfg` (persisted last effective root).
- Runtime view sidecars: per-directory `.mdexplore-views.json` files for persisted view-tab state (multi-view sessions and custom tab labels).
- Runtime preview-highlight sidecars: per-directory `.mdexplore-highlighting.json` files for persistent text highlights created from the preview pane.

Runtime Assumptions

- Python `3.10+`.
- Linux desktop with GUI support.
- `PySide6` and `QtWebEngine` available via pip dependencies.
- `pypdf` and `reportlab` available for PDF page-number stamping.
- Mermaid should prefer a local bundle when available and only use CDN fallback.
- Rust Mermaid backend is optional and uses `mmdr` when `--mermaid-backend rust` is selected.
- MathJax should prefer a local bundle when available and only use CDN fallback.
- Java runtime is expected for local PlantUML rendering.
- VS Code `code` command may or may not be installed; app should fail gracefully when missing.

Core Behavior You Must Preserve

- Without a path arg, default root is loaded from `~/.mdexplore.cfg` if valid; otherwise home directory is used.
- Both entrypoints support optional root path argument:
  - `mdexplore.sh [PATH]`
  - `mdexplore.py [PATH]`
- Tree shows directories and `.md` files only.
- Selecting a Markdown file updates preview quickly.
- GitHub/Obsidian-style markdown callouts (`> [!TYPE]`) should render as styled callout boxes in preview and PDF output.
- Navigating to a previously cached file must still stat-check mtime/size; if changed, it must re-render instead of showing stale cached HTML.
- PlantUML rendering jobs must run off the UI thread to keep the window responsive.
- PlantUML blocks should not block markdown preview; show placeholders first, then progressively replace each diagram as local render jobs complete.
- PlantUML progressive updates should be applied in-place so the preview scroll location is preserved.
- PlantUML progress should persist across file navigation so returning to a file shows completed diagrams without restarting work.
- Preview scroll position should be remembered per markdown file for the current application run.
- While dragging the preview pane's vertical scrollbar, the preview should show an approximate `current line / total lines` indicator beside the scrollbar handle.
- `Ctrl++`, `Ctrl+-`, and `Ctrl+0` should adjust only the preview pane zoom factor, not the tree pane or overall UI scale.
- Preview zoom changes should briefly show a top-center percentage overlay inside the preview area.
- `^` navigates one directory level up and re-roots the tree.
- User-adjusted splitter width between tree and preview should persist across root changes for the current application run.
- Window title reflects effective root scope (selected directory if selected, otherwise current root).
- Effective root is persisted on close to `~/.mdexplore.cfg`.
- Linux desktop identity should remain `mdexplore` so launcher icon matching works (`QApplication.setDesktopFileName("mdexplore")` + desktop `StartupWMClass=mdexplore`).
- `Edit` opens currently selected file with `code`.
- `Add View` creates another tabbed view of the same current document, starting from the current top-visible line/scroll position.
- View tabs default to top-visible source line numbers, are closeable, and are capped at 8 tabs per document.
- View tabs include a left-side mini bargraph indicating each view's position in the current document.
- View tabs should keep a stable soft-pastel color sequence based on open order.

- View tabs are draggable/reorderable; moving tabs must not change their assigned color.
- Right-clicking a view tab should allow editing a custom tab label up to 48 characters, including spaces.
- If a longer tab label is entered, it should be truncated to the first 48 characters rather than rejected.
- If a custom tab label is cleared back to blank, that tab should resume using the default dynamic line-number label.
- When a tab receives a custom label, the app should capture that tab's current scroll position and top visible source line as the saved label-time beginning.
- Right-clicking a custom-labeled tab should also offer `Return to beginning`, which restores that saved label-time location for the tab.
- Relabeling a custom-labeled tab with different text should reset the saved beginning to the tab's current scroll position/top line at relabel time.
- When assigning a new tab color and wrapping the palette, skip any color already used by open tabs.
- Per-document tabs should persist for the app run: switching to another markdown file and back must restore prior tabs, their order, and the previously selected tab.
- View-tab state should also persist across app restarts via per-directory `.mdexplore-views.json`, keyed by markdown filename.
- For custom-labeled tabs, `.mdexplore-views.json` should also persist the saved label-time beginning location used by `Return to beginning`.
- If custom labels make the tab strip wider than the available space, the tab bar should scroll rather than truncating the tab set.
- Only documents with explicit multi-view state or custom tab labels should be written to `.mdexplore-views.json`; untouched single-view documents should fall back to the default one-tab state without sidecar entries.
- The tab strip should remain hidden when there is only one unlabeled default view.
- If only one view remains and it has a custom label, its tab should stay visible so the custom label and `Return to beginning` action remain available.
- Closing that sole remaining custom-labeled tab should clear the custom label and saved beginning, then fall back to the hidden default single-view state.
- `Refresh` button and `F5` both refresh the current directory tree view (new files appear, deleted files disappear) without requiring app restart.
- `PDF` exports the currently previewed markdown rendering to `<source-basename>.pdf` beside the source file, with centered `N of M` numbering at the bottom of each page.
- PDF export should preflight preview readiness (MathJax/Mermaid/fonts) before snapshotting, and apply print-focused math styling to avoid cramped glyphs.
- `MIN_PRINT_DIAGRAM_FONT_PT` in `mdexplore.py` is the tweakable PDF one-page shrink floor for diagram sizing decisions. Lowering it allows tall diagrams (for example Mermaid activity/flowcharts) to stay on one page more aggressively before the exporter falls back to multi-page spill.
- In PDF mode, headed Mermaid and PlantUML sections should attach the heading to the diagram fence itself for pagination purposes, so Chromium cannot leave the heading on one page and push

the diagram to the next.
- If the currently previewed markdown file changes on disk, preview should auto-refresh and report that via status bar message.
- Status bar should show progress during long-running operations (preview loading/rendering, PlantUML completion, PDF export) and should not remain blank during idle gaps.
- Search input uses label `Search and highlight:` and includes an explicit in-field `X` clear action.
- Pressing `Enter` in search should bypass debounce and run search immediately.
- Non-quoted search terms should be case-insensitive; quoted terms should be case-sensitive.
- `CLOSE(...)` should be supported in search queries and require all listed terms to occur within `SEARCH_CLOSE_WORD_GAP` content words.
- Function-style operators should accept both no-space and spaced forms before `(` (for example `OR(...)` and `OR (...)`).
- `AND(...)`, `OR(...)`, and `CLOSE(...)` should accept comma-delimited, space-delimited, or mixed argument lists.
- `AND(...)` and `OR(...)` should be variadic (2+ arguments).
- If search is active and directory scope changes, search should rerun against the new directory scope.
- File highlight colors are assigned from tree context menu and persisted per directory.
- Highlight state persists in `.mdexplore-colors.json` files where writable.
- `Clear All` in the context menu recursively removes highlight metadata after confirmation.
- Copy/Clear scope resolves in this order: selected directory, then most recently selected/expanded directory, then current root.
- Clipboard copy must preserve Nemo/Nautilus compatibility (`text/uri-list` plus `x-special/gnome-copied-files`).
- The pin copy action should copy exactly the currently previewed markdown file using the same clipboard MIME compatibility guarantees.
- Preview context menu should offer persistent `Highlight` / `Remove Highlight` actions for rendered text selections and existing highlighted blocks.
- Preview text highlights should persist per directory in `.mdexplore-highlighting.json`.
- Persistent preview text highlights should also surface as purple left-gutter markers in the preview; clicking a marker should jump to the corresponding highlighted block, and taller markers should reflect highlights that span more lines.
- Preview context menu should keep standard actions and add `Copy Rendered Text` and `Copy Source Markdown` when there is a text selection.
- `Copy Rendered Text` should copy the selected preview text as plain text.
- `Copy Source Markdown` should map preview selection to source markdown line ranges and copy source text (not rendered plain text). If direct mapping fails, it should use selected-text matching and first/last line fuzzy matching against source markdown lines, then fall back to copying the full source file.
- While search is active, opening a matched markdown file should highlight matched terms in yellow in preview and scroll to the first match.

- While search is active, the preview should show yellow scrollbar-side markers for highlighted hits, and clicking a marker should jump to the nearest underlying hit represented by that marker cluster.
- While search is active, matching markdown files in the tree should show a yellow hit-count pill in the left gutter rather than the older triangle badge.
- Markdown files that currently have more than the default single view should show a small light-gray tab badge beside the markdown icon in the tree.
- Markdown files with persisted preview text highlights should also show a separate marker badge in the tree.
- Tree gutter badge order for markdown files is: search hit-count pill, highlight marker, views badge, markdown file icon.
- The markdown file icon and filename must stay horizontally aligned across all markdown rows by using a fixed-width gutter strip and packing only the active badges inside it.
- Named views with saved `Return to beginning` anchors should also appear as color-matched left-side preview gutter markers positioned by their saved home line number. These markers must render above persistent-highlight markers when their positions overlap.

Editing Rules

- Keep code ASCII unless file already requires Unicode.
- Prefer type hints and straightforward, explicit control flow.
- Avoid large framework changes unless requested.
- Do not add heavy dependencies without clear user value.
- Keep startup and preview interactions responsive.
- Preserve cache semantics unless changing performance behavior intentionally.

Rendering Rules

- Markdown parser is `markdown-it-py`.
- Mermaid and MathJax are rendered client-side in web view.
- Mermaid backend is switchable via CLI:
  - `--mermaid-backend js` (default)
  - `--mermaid-backend rust` (renders SVG with `mmdr`)
- Mermaid loading order is local-first, then CDN fallback.
- Mermaid rendering should use a dark-theme-friendly high-contrast palette when preview background is dark.
- Mermaid SVG render results should be cached in-memory (by diagram hash and render mode) for the current app run so revisiting documents avoids rerender.
- Mermaid cache modes are:
  - `auto`: GUI/interactive preview mode.
  - `pdf`: print/export mode.
- PDF Mermaid behavior is backend-specific:
  - JS backend: uses PDF clean render + print-safe monochrome/grayscale transform.
  - Rust backend: uses separate PDF SVGs rendered by `mmdr` with default theming, then applies print-safe grayscale normalization (with gradation and readable text). GUI preview post-processing must not leak into Rust PDF source SVG selection.
- `MDEXPLORE_MERMAID_JS` can be used to force a specific local Mermaid script path.
- `MDEXPLORE_MERMAID_RS_BIN` can be used to force a specific `mmdr` executable path.
- MathJax loading order is local-first, then CDN fallback.
- `MDEXPLORE_MATHJAX_JS` can be used to force a specific local MathJax script path.
- Callout syntax is parsed from blockquotes using `[!TYPE]` markers and rendered as non-collapsible callout containers.
- PlantUML fences are rendered locally through `plantuml.jar` (`java -jar ...`), defaulting to `vendor/plantuml/plantuml.jar` unless `PLANTUML_JAR` is set.
- PlantUML failures should render inline as: `PlantUML render failed with error ...`, including detailed stderr context (line numbers when available).
- Maintain base URL behavior (`setHtml(..., base_url)`) so relative links/images resolve.
- If adding new embedded syntaxes, implement via fenced code handling and document it.
- Debug logging to project-root `mdexplore.log` must remain disabled unless the app is launched with `--debug`.

Mermaid Render/Caching Architecture (Read Before Changes)

This code has multiple render forks. Preserve these boundaries. For practical, symptom-driven troubleshooting, also read section `10. Debugging Playbook (Human + Agent)` in `RENDER-PATHS.md`.

Python-side responsibilities

- `MarkdownRenderer` parses markdown fences and renders Mermaid differently by backend.
- When backend is `rust`:
  - Preview SVG is rendered using Rust preview profile and embedded in HTML.
  - A second PDF-profile SVG (default `mmdr` theming) is rendered per Mermaid hash and stored in `env["mermaid_pdf_svg_by_hash"]`.
- `render_document()` captures this per-document Rust PDF map and exposes it through `take_last_mermaid_pdf_svg_by_hash()`.
- `MdExploreWindow._merge_renderer_pdf_mermaid_cache_seed()` merges that map into runtime Mermaid cache mode `pdf`.
- `_inject_mermaid_cache_seed()` injects the full mode cache (`auto` + `pdf`) into page JS.

In-page JS responsibilities

- `window.__mdexploreMermaidSvgCacheByMode` stores mode-specific SVG caches.
- GUI path:
  - JS backend renders Mermaid through Mermaid JS and stores SVGs in `auto`.
  - Rust backend prefers pre-rendered Rust SVG already in DOM; if missing, may fallback to Mermaid JS (unless explicitly disabled for that operation).
  - GUI post-processing/tuning is applied only in GUI mode.
- PDF preflight path:
  - JS backend runs PDF-mode Mermaid render and then monochrome/grayscale transform.
  - Rust backend swaps `.mermaid` blocks to cached Rust `pdf` SVGs and then applies the same print-safe monochrome/grayscale normalization pass.

Critical invariant

- Never reuse GUI-adjusted Mermaid SVG as Rust PDF output.
- Never select Mermaid cache mode `auto` during Rust PDF export.
- Never leave PDF-mode Rust SVG in place after export completes.
- Restore path (`_restore_preview_mermaid_palette`) must force mode `auto` reapplication. For Rust, this means replacing with cached `auto` SVGs (or regenerating if unavailable).

Practical maintenance guidance

- If you change Mermaid DOM wrappers/toolbars/viewport behavior:
  - Re-test GUI render, PDF export, and post-PDF return for both JS and Rust backends.
- If you change cache keys:
  - Keep mode separation (`auto` vs `pdf`) and hash normalization stable.
- If you change preprint normalization:
  - Ensure toolbars/scrollbars are hidden in PDF.
  - Ensure width/height normalization does not break GUI restore.

Known TODO: Diagram View State Restore

- Open issue: Mermaid and PlantUML zoom/pan state is not yet reliably restored when switching away from a document and then returning during the same app run.
- Treat this as a known TODO. Do not remove this note until behavior is verified end-to-end and documented with a reproducible test.

Attempts Tried That Did Not Fully Resolve It

- Frequent JS-side state capture timers plus explicit capture on file-switch.
- Per-diagram state keys and in-page reapply hooks (`__mdexploreReapplySavedState`).
- Multi-pass deferred restore reapply from Python (`QTimer` at staggered delays).
- Reapplying restore after Mermaid palette reset and after async renderer completion.
- Forcing PDF-mode normalization/reset before print and restoring interaction state afterward.

These approaches improved behavior in some cases but did not make restore reliable across all navigation sequences.

Launcher Rules

- `mdexplore.sh` must be runnable from any working directory.
- It must keep venv handling deterministic:
  - create if missing
  - install dependencies
  - run app
  - invoke `.venv/bin/python` directly (no shell activation required)
  - do not use `exec -a` with Python (can break venv resolution)
- Launcher arg handling must tolerate `.desktop` `%u` behavior:
  - empty argument should behave like "no path"
  - `file://` URIs should be decoded
  - file arguments should resolve to parent directory
- Launcher must pass through supported app switches (for example `--mermaid-backend`).
- Launcher `DEBUG_MODE=true` should append `--debug` so application debug logging can be enabled without editing desktop launchers or shell aliases.
- Non-interactive launcher runs should log to `~/.cache/mdexplore/launcher.log` for desktop troubleshooting. Log retention is capped to the most recent 1000 lines.
- Launcher should verify key runtime imports (`markdown_it` , `linkify_it` , `PySide6.QtWebEngineWidgets` , `pypdf` , `reportlab.pdfgen.canvas`) and self-heal by reinstalling requirements if the environment is incomplete.
- `--help` should stay lightweight and not require venv activation.

Quality Gates Before Finishing

Run at least:

```
bash -n mdexplore.sh
python3 -m py_compile mdexplore.py
```

If behavior changes, also run manual smoke tests:

1. Launch with no path.
2. Launch with explicit path.
3. Open `.md` file and verify render.
4. Verify `Edit` behavior with and without `code` in `PATH`.

Documentation Requirements

When changing behavior:

- Update `README.md` usage and examples.
- Update `RENDER-PATHS.md` when render/caching/PDF/diagram control flow changes.
- Update this file if agent-facing workflow or guarantees changed.

Common Feature Patterns

- New keyboard action: add `QAction` in `_add_shortcuts`.
- New toolbar action: add button in top bar setup and handler method.
- New markdown extension: extend custom fence handling in `MarkdownRenderer`.
- New CLI option: update argparse setup in `main`, then mirror wrapper behavior in `mdexplore.sh`.

Out of Scope Unless Requested

- Packaging into Snap/Deb/AppImage.
- Multi-tab editing or embedded text editor.
- Background file watching/auto-reload loops.
- Cross-platform polish outside Linux desktop behavior.